

HiDiTingV100 安全软件

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要描述了安全驱动相关结构、软件接口的使用方法以及注意事项，用于指导客户进行安全驱动相关开发。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
 须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 概述.....1

1.1 功能描述..... 1

1.1.1 KM 2

1.1.2 AES..... 2

1.1.3 HASH..... 2

1.1.4 RSA 3

1.1.5 TRNG..... 3

1.2 使用流程..... 3

1.2.1 密钥配置..... 3

1.2.1.1 工作流程..... 3

1.2.1.2 代码示例..... 4

1.2.2 AES 加解密..... 6

1.2.2.1 工作流程..... 6

1.2.2.2 代码示例..... 7

1.2.2.3 注意事项..... 13

1.2.3 HASH 计算..... 14

1.2.3.1 工作流程..... 14

1.2.3.2 代码示例..... 14

1.2.3.3 注意事项..... 15

1.2.4 RSA 加解密..... 15

1.2.4.1 工作流程..... 15

1.2.4.2 代码示例..... 16

1.2.4.3 注意事项..... 22

1.2.5 RSA 签名验签..... 22

1.2.5.1 工作流程.....	22
1.2.5.2 代码示例.....	22
1.2.5.3 注意事项.....	23
1.2.6 大数模幂运算	24
1.2.6.1 工作流程.....	24
1.2.6.2 代码示例.....	24
1.2.6.3 注意事项.....	27
1.2.7 TRNG.....	27
1.2.7.1 工作流程.....	27
1.2.7.2 代码示例.....	27
2 API 参考.....	28
2.1 KM API	28
2.1.1 uapi_drv_klad_create	28
2.1.2 uapi_drv_klad_destroy	29
2.1.3 uapi_drv_klad_attach	29
2.1.4 uapi_drv_klad_detach	30
2.1.5 uapi_drv_klad_set_attr	31
2.1.6 uapi_drv_klad_get_attr	32
2.1.7 uapi_drv_klad_set_clear_key	33
2.1.8 uapi_drv_keyslot_create.....	33
2.1.9 uapi_drv_keyslot_destroy.....	34
2.2 CIPHER API.....	35
2.2.1 uapi_drv_cipher_symc_create.....	35
2.2.2 uapi_drv_cipher_symc_destroy.....	36
2.2.3 uapi_drv_cipher_symc_set_config	36
2.2.4 uapi_drv_cipher_symc_get_config	37
2.2.5 uapi_drv_cipher_symc_attach.....	38
2.2.6 uapi_drv_cipher_symc_detach.....	39
2.2.7 uapi_drv_cipher_symc_encrypt.....	39
2.2.8 uapi_drv_cipher_symc_decrypt.....	41
2.2.9 uapi_drv_cipher_symc_get_tag.....	42
2.3 HASH API	43
2.3.1 uapi_drv_cipher_hash_start	43
2.3.2 uapi_drv_cipher_hash_update	43
2.3.3 uapi_drv_cipher_hash_final.....	44
2.4 RSA API	45
2.4.1 uapi_drv_cipher_pke_rsa_public_encrypt.....	45
2.4.2 uapi_drv_cipher_pke_rsa_private_decrypt	46

2.4.3 uapi_drv_cipher_pke_rsa_sign.....	47
2.4.4 uapi_drv_cipher_pke_rsa_verify.....	48
2.4.5 uapi_drv_cipher_pke_exp_mod	49
2.5 TRNG API	50
2.5.1 uapi_drv_cipher_trng_get_random.....	50
2.6 错误码	51
3 数据类型说明	53
3.1 KM.....	53
3.1.1 drv_klad_attr_t.....	53
3.1.2 drv_klad_config_t.....	54
3.1.3 drv_klad_clear_key_t.....	54
3.2 AES.....	55
3.2.1 cipher_work_mode_t	55
3.2.2 cipher_config_aes_t.....	56
3.2.3 cipher_config_aes_gcm_t.....	57
3.2.4 cipher_config_aes_ccm_t.....	58
3.3 HASH.....	59
3.3.1 drv_cipher_hash_type_t	59
3.3.2 drv_cipher_hash_attr_t.....	61
3.3.3 drv_cipher_buf_attr_t.....	61
3.4 RSA.....	62
3.4.1 drv_pke_len_t	62
3.4.2 drv_pke_data_t.....	62
3.4.3 drv_pke_rsa_scheme_t	63
3.4.4 drv_pke_rsa_pub_key_t	64
3.4.5 drv_pke_rsa_priv_key_t	64

插图目录

图 1-1 CIPHER DRIVER 一级设计图 2

Draft

1 概述

1.1 功能描述

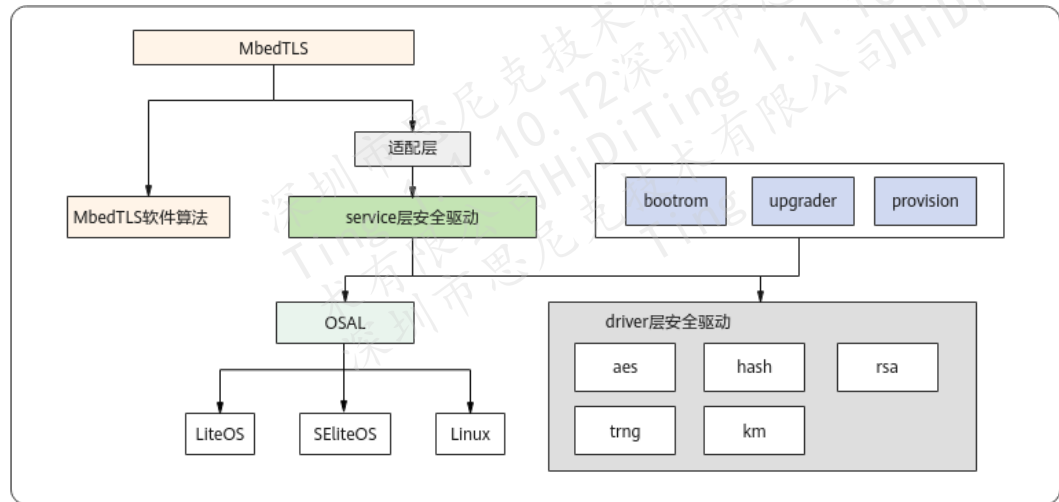
1.2 使用流程

1.1 功能描述

CIPHER DRIVER 为安全算法模块，包括两层接口：service driver 和底层 driver，如图 1-1 所示。

- service driver
 - 为上层应用提供加解密能力，可由 mbedtls 等调用。
 - 在 rom driver 基础上进一步封装，支持 wait_event 和中断轮询模式（默认开启 wait_event 模式）。
- 底层 driver
 - 主要由 bootrom、upgrader、provision 等使用，在无 OS 环境下提供加解密能力。
 - 通过配置寄存器的方式调用硬件逻辑计算能力，对外提供统一调用接口。

图1-1 CIPHER DRIVER 一级设计图



CIPHER DRIVER 提供了 AES 对称加解密算法、HASH 摘要算法、RSA 非对称加解密及签名验签算法、随机数算法以及密钥管理。主要应用于 AIOT 产品安全启动、安全存储、安全通讯等场景。

CIPHER DRIVER 包括 5 个子模块：KM、AES、HASH、RSA、TRNG。

1.1.1 KM

KM (Key Management) 即密钥管理模块，可用于增加密钥安全强度，将 key 加载到 keyslot 中。支持硬件 key 和软件 key。

1.1.2 AES

AES：支持 ECB/CBC/OFB/CTR/CCM/GCM 共 6 种工作模式。其中：

- OFB 模式支持的加密数据位宽可为 128bit。
- CCM、GCM 模式在解密结束后需获取一次 tag 值。
- ECB、CBC 模式加密数据长度必须为 16 字节对齐。

须知

ECB 模式属于非安全算法，不建议使用。

1.1.3 HASH

HASH 即散列函数，可用于验证数据的完整性。支持规格如下：

- 仅支持 SHA256。
- 支持中途传入非 block size 对齐的数据块。
- 支持重试机制，即中间过程调用接口出错可重新调用该接口直至成功，而不需要从头开始整个计算流程，具体内容参见 uapi_drv_cipher_hash_update 接口。

1.1.4 RSA

RSA 即非对称加解密算法。常用于文件的签名和验签，以及对对称密钥的加解密。

- 支持 PKCS#1 v1.5 和 PKCS#1 v2.1 PSS 两种模式的私钥签名和公钥验签。
- 支持 PKCS#1 v1.5 和 PKCS#1 v2.1 OAEP 两种模式的公钥加密和私钥解密。
- 支持密钥位宽 1024/2048/4096 bit。
- 支持 1024/2048/4096 bit 的模幂运算。

须知

- RSA1024 位不安全，不建议使用。
- PKCS#1 v1.5 填充方式不安全，不建议使用。

1.1.5 TRNG

随机数获取模块，获取硬件产生的真随机数。

1.2 使用流程

1.2.1 密钥配置

1.2.1.1 工作流程

步骤 1 调用 uapi_drv_keyslot_create 接口创建 keyslot 通道。

步骤 2 调用 uapi_drv_klad_create 接口创建一路 klad (key ladder)，并获取 klad 句柄。

步骤 3 调用 uapi_drv_klad_attach 接口绑定 keyslot 通道与 klad 句柄。（步骤 3、步骤 4 的顺序可调换）。

步骤 4 调用 uapi_drv_klad_set_attr 接口配置 klad 属性。（请参见“1.2.1.2 代码示例”）

步骤 5 调用 uapi_drv_klad_set_clear_key 接口配置明文 key。

步骤 6 调用 uapi_drv_klad_detach 接口解绑 keyslot 与 klad 句柄。

步骤 7 调用 uapi_drv_klad_destroy 接口销毁 klad 句柄。

步骤 8 调用 uapi_drv_keyslot_destroy 接口销毁 keyslot 句柄。

----结束

1.2.1.2 代码示例

```
uint8_t g_aes_key[] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C,
};

errcode_t cipher_set_key_test(void)
{
    errcode_t ret;
    uint32_t klad;
    uint32_t check_word;
    drv_klad_clear_key_t clear_key;
    uapi_drv_klad_dest_t dest = DRV_KLAD_DEST_TYPE_MCIPHER;
    drv_klad_attr_t attr = { 0 };

    attr.key_cfg.engine      = DRV_CRYPTTO_ALG_AES;
    attr.key_cfg.decrypt_support = 0x1;
    attr.key_cfg.encrypt_support = 0x1;

    clear_key.key_size      = AES_KEY_128BIT;
    clear_key.key           = g_aes_key;

    // 创建keyslot
    uint32_t keyslot_handle = INVALID_HANDLE;
    ret = uapi_drv_keyslot_create(&keyslot_handle, DRV_KEYSLLOT_TYPE_MCIPHER);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    // 创建klad
    ret = uapi_drv_klad_create(&klad);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
}
```

```
// 绑定klad到keyslot

check_word = (uint32_t) klad ^ (uint32_t) dest ^ (uint32_t) keyslot_handle;
ret = uapi_drv_klad_attach(klad, dest, keyslot_handle, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 设置klad属性

check_word = (uint32_t) klad ^ (uint32_t) &attr;
ret = uapi_drv_klad_set_attr(klad, &attr, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 设置密钥

check_word = (uint32_t) klad ^ (uint32_t) &clear_key;
ret = uapi_drv_klad_set_clear_key(klad, &clear_key, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 解绑klad与keyslot

check_word = (uint32_t) klad ^ (uint32_t) dest ^ (uint32_t) keyslot_handle;
ret = uapi_drv_klad_detach(klad, dest, keyslot_handle, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 销毁klad

ret = uapi_drv_klad_destroy(klad);
if(ret != ERRCODE_SUCC) {
    return ret;
}

// 销毁keyslot

ret = uapi_drv_keyslot_destroy(keyslot_handle);
if(ret != ERRCODE_SUCC) {
    return ret;
}
```

```
return ERRCODE_SUCC;

exit:
(void)uapi_drv_klad_destroy(klad);
(void)uapi_drv_keyslot_destroy(keyslot_handle);
return ERRCODE_FAIL;
}
```

1.2.2 AES 加解密

1.2.2.1 工作流程

步骤 1 调用 uapi_drv_cipher_symc_create 接口创建一路 cipher，并获取 cipher 句柄。

步骤 2 调用 uapi_drv_keyslot_create 接口创建 keyslot 通道。

步骤 3 调用 KM 模块，配置加解密密钥至 keyslot 通道。请参见“[1.2.1.1 工作流程](#)”小节中[步骤 2~步骤 7](#)。

步骤 4 调用 uapi_drv_cipher_symc_attach 接口绑定 keyslot 通道与 cipher 句柄。（[步骤 3](#)、[步骤 4](#) 的顺序可调换）。

步骤 5 调用接口 uapi_drv_cipher_symc_set_config 接口配置 cipher 控制信息。（请参见“[1.2.2.2 代码示例](#)”）

步骤 6 对数据进行加解密。用户可以调用以下对应接口进行加解密：

- 加密：uapi_drv_cipher_symc_encrypt。
- 解密：uapi_drv_cipher_symc_decrypt。

步骤 7 如果模式为 CCM 和 GCM，调用加解密接口后需要继续调用 uapi_drv_cipher_symc_get_tag 接口获取 tag 值，用于认证结果判断，否则直接执行[步骤 8](#)。

步骤 8 调用 uapi_drv_cipher_symc_detach 接口解绑 keyslot 通道与 cipher 句柄。

步骤 9 调用 uapi_drv_keyslot_destroy 接口销毁 keyslot 通道。

步骤 10 调用 uapi_drv_cipher_symc_destroy 接口销毁 cipher 句柄。

----结束

1.2.2.2 代码示例

```
uint8_t g_aes_key[] = {
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3C,
    0x2B, 0x7E, 0x15, 0x16, 0x28, 0xAE, 0xD2, 0xA6,
    0xAB, 0xF7, 0x15, 0x88, 0x09, 0xCF, 0x4F, 0x3D
};

uint8_t g_aes_iv[] = {
    0x00, 0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07,
    0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F
};

uint8_t g_aes_src[] = {
    0x6B, 0xC1, 0xBE, 0xE2, 0x2E, 0x40, 0x9F, 0x96,
    0xE9, 0x3D, 0x7E, 0x11, 0x73, 0x93, 0x17, 0x2A,
    0x6B, 0xC1, 0xBE, 0xE2, 0x2E, 0x40, 0x9F, 0x96,
    0xE9, 0x3D, 0x7E, 0x11, 0x73, 0x93, 0x17, 0x2A
};

typedef struct {
    drv_cipher_buf_attr_t src;
    uint32_t src_len;
    drv_cipher_buf_attr_t dst;
    uint32_t dst_len;
    uint8_t *iv;
    uint32_t iv_len;
    uint8_t *key;
    uint32_t key_len;
    uint8_t *expect_tag;
    uint32_t expect_tag_len;
    drv_cipher_alg_t crypto_alg;
    union {
        drv_cipher_config_aes_t aes_config;
        drv_cipher_config_aes_gcm_t aes_gcm_config;
        drv_cipher_config_aes_ccm_t aes_ccm_config;
    } cipher_config_t;
    drv_cipher_attr_t cipher_attr;
    drv_cipher_config_t ctrl;
    bool is_clear_key;
} cipher_test_info_t;
```

```

cipher_test_info_t g_aes_test_info = {
    .src = {
        .address = g_aes_src,
        .buf_sec = DRV_CIPHER_BUF_NONSECURE
    },
    .src_len = sizeof(g_aes_src),
    .dst = {
        .address = NULL,
        .buf_sec = DRV_CIPHER_BUF_NONSECURE
    },
    .iv = g_aes_iv,
    .iv_len = sizeof(g_aes_iv),
    .key = g_aes_key,
    .key_len = AES_KEY_128BIT,
    .expect_tag = NULL,
    .expect_tag_len = 0,
    .crypto_alg = DRV_CIPHER_ALG_AES, // needed or can be replaced by compare other
parameters
    .cipher_config_t.aes_config = {
        .key_len = DRV_CIPHER_KEY_128BIT,
        .bit_width = DRV_CIPHER_BIT_WIDTH_128BIT,
        .iv_change_flag = DRV_CIPHER_IV_CHANGE_ONE_PKG,
        .key_parity = false
    },
    .cipher_attr = {
        .cipher_type = DRV_CIPHER_TYPE_NORMAL,
    },
    .ctrl = {
        .alg = DRV_CIPHER_ALG_AES,
        .work_mode = DRV_CIPHER_WORK_MODE_CBC,
        .dfa = DRV_CIPHER_DFA_DISABLE
    },
    .is_clear_key = false,
};

// 设置key

static errcode_t cipher_set_clear_key(uint32_t keyslot, unsigned char *key, const unsigned int
keylen,

                                const drv_klad_key_parity_t key_parity, const
drv_cipher_alg_t alg)
{
    errcode_t ret;
    uint32_t klad;

```

```
uint32_t check_word;
drv_klad_clear_key_t clear_key;
uapi_drv_klad_dest_t dest = DRV_KLAD_DEST_TYPE_MCIPHER;
drv_klad_attr_t attr = { 0 };

attr.key_cfg.engine      = (drv_crypto_alg_t)alg;
attr.key_cfg.decrypt_support = 0x1;
attr.key_cfg.encrypt_support = 0x1;

clear_key.key_size      = keylen; /* 16 or 32 */
clear_key.key           = key;
clear_key.key_parity    = key_parity;

// 创建klad

ret = uapi_drv_klad_create( &klad );
if(ret != ERRCODE_SUCC) {
    return ret;
}

// 绑定klad到keyslot

check_word = (uint32_t) klad ^ (uint32_t) dest ^ (uint32_t) keyslot;
ret = uapi_drv_klad_attach(klad, dest, keyslot, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 设置klad属性

check_word = (uint32_t) klad ^ (uint32_t) &attr;
ret = uapi_drv_klad_set_attr(klad, &attr, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 设置密钥

check_word = (uint32_t) klad ^ (uint32_t) &clear_key;
ret = uapi_drv_klad_set_clear_key(klad, &clear_key, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 解绑klad与keyslot
```



```
check_word = (uint32_t) klad ^ (uint32_t) dest ^ (uint32_t) keyslot;
ret = uapi_drv_klad_detach(klad, dest, keyslot, check_word);
if(ret != ERRCODE_SUCC) {
    goto exit;
}

// 销毁klad
ret = uapi_drv_klad_destroy(klad);
if(ret != ERRCODE_SUCC) {
    return ret;
}

return ERRCODE_SUCC;

exit:
(void)uapi_drv_klad_destroy(klad);
return ERRCODE_FAIL;
}

// cipher通用处理
errcode_t cipher_common(cipher_test_info_t *test_info)
{
    errcode_t ret;
    uint32_t symc_handle = INVALID_HANDLE;
    uint32_t keyslot_handle = INVALID_HANDLE;

    uint32_t check_word;
    uint32_t length;
    drv_cipher_buf_attr_t src_buf;
    drv_cipher_buf_attr_t dest_buf;
    static uint8_t result[DATA_LEN] __attribute__((aligned (CACHE_LINE_SIZE))) = {0};

    src_buf.address = test_info->src.address;
    src_buf.buf_sec = test_info->src.buf_sec;
    dest_buf.address = result;
    dest_buf.buf_sec = test_info->dst.buf_sec;
    length = test_info->src_len;

    // 创建cipher通道
    check_word = (uint32_t)(&symc_handle) ^ (uint32_t)(&test_info->cipher_attr);
    ret = uapi_drv_cipher_symc_create(&symc_handle, &test_info->cipher_attr, check_word);
    if (ret != ERRCODE_SUCC) {
```

```
        return ret;
    }

    // 创建keyslot

    ret = uapi_drv_keyslot_create(&keyslot_handle, DRV_KEYSLLOT_TYPE_MCIPHER);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    // 绑定cipher与keyslot

    check_word = (uint32_t)(symc_handle) ^ (uint32_t)keyslot_handle;
    ret = uapi_drv_cipher_symc_attach(symc_handle, keyslot_handle, check_word);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    // 设置cipher加密属性

    check_word = (uint32_t)(symc_handle) ^ (uint32_t)&(test_info->ctrl);
    ret = uapi_drv_cipher_symc_set_config(symc_handle, &test_info->ctrl, check_word);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    // 设置密钥

    ret = cipher_set_clear_key(keyslot_handle, test_info->key, test_info->key_len,
        (drv_klad_key_parity_t)((drv_cipher_config_aes_t*)test_info->ctrl.param->key_parity,
test_info->crypto_alg);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    // 加密

    check_word = (uint32_t)symc_handle ^ (uint32_t)&(src_buf) ^ (uint32_t)&(dest_buf) ^
(uint32_t)length ^ 0;
    ret = uapi_drv_cipher_symc_encrypt(symc_handle, &src_buf, &dest_buf, length, 0,
check_word);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    if (test_info->dst.address != NULL) {
```

```
ret = memcmp(test_info->dst.address, result, length); // change for use input parama
if (ret != ERRCODE_SUCC) {
    return ret;
}

// 设置cipher解密属性
check_word = (uint32_t)(symc_handle) ^ (uint32_t)(test_info->ctrl);
ret = uapi_drv_cipher_symc_set_config(symc_handle, &test_info->ctrl, check_word);
if (ret != ERRCODE_SUCC) {
    return ret;
}

src_buf.address = test_info->dst.address;
src_buf.buf_sec = test_info->dst.buf_sec;
dest_buf.address = result;
dest_buf.buf_sec = test_info->src.buf_sec;

// 解密
check_word = (uint32_t)symc_handle ^ (uint32_t)(src_buf) ^ (uint32_t)(dest_buf) ^
(uint32_t)length ^ 0;
ret = uapi_drv_cipher_symc_decrypt(symc_handle, &src_buf, &dest_buf, length, 0,
check_word);
if (ret != ERRCODE_SUCC) {
    return ret;
}

if (test_info->src.address != NULL) {
    if (memcmp(test_info->src.address, result, length) != 0) {
        return ERRCODE_FAIL;
    }
}

// 销毁相关资源
check_word = (uint32_t)(symc_handle) ^ (uint32_t)keyslot_handle;
(void)uapi_drv_cipher_symc_detach(symc_handle, keyslot_handle, check_word);
(void)uapi_drv_keyslot_destroy(keyslot_handle);
(void)uapi_drv_cipher_symc_destroy(symc_handle);

return ret;
}
```

```

static errcode_t aes_param_config(cipher_test_info_t *aes_test_info)
{
    if (aes_test_info->ctrl.work_mode != DRV_CIPHER_WORK_MODE_ECB) {
        memcpy_s(aes_test_info->cipher_config.t.aes_config.iv,
        DRV_CIPHER_AES_IV_LEN_IN_BYTES,
        aes_test_info->iv, aes_test_info->iv_len);
    }
    aes_test_info->ctrl.param = &(aes_test_info->cipher_config.t.aes_config);
    return cipher_common(aes_test_info);
}

// 以cbc模式测试为例
errcode_t cipher_aes_cbc_test(void)
{
    errcode_t ret;
    static uint8_t aes_dst[] = {
        0xC3, 0xBE, 0x4C, 0xEF, 0x2C, 0x67, 0xBC, 0x8B,
        0x4B, 0x91, 0x2F, 0x85, 0x1C, 0xBD, 0x09, 0x8F,
        0xF3, 0xCF, 0x17, 0x91, 0x4F, 0x12, 0x34, 0x49,
        0xB7, 0xEE, 0xEF, 0x47, 0x81, 0x6F, 0x24, 0x92
    };

    cipher_test_info_t aes_cbc_test_info;
    (void)memcpy_s(&aes_cbc_test_info, sizeof(aes_cbc_test_info), &g_aes_test_info,
    sizeof(g_aes_test_info));

    aes_cbc_test_info.dst.address = aes_dst;
    aes_cbc_test_info.dst_len = sizeof(aes_dst);
    aes_cbc_test_info.key_len = DRV_CIPHER_KEY_256BIT;
    aes_cbc_test_info.cipher_config.t.aes_config.key_len = DRV_CIPHER_KEY_256BIT;

    ret = aes_param_config(&aes_cbc_test_info);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }

    return ERRCODE_SUCC;
}

```

1.2.2.3 注意事项

支持 AES ECB/CBC/OFB/CTR/CCM/GCM 共 6 种模式。

对于 ECB/CBC/OFB/CTR 模式，支持重试，即在调用 uapi_drv_cipher_symc_encrypt/uapi_drv_cipher_symc_decrypt 接口失败时，仍可配置正确参数后继续重新调用该接口，完成后续数据加解密。

1.2.3 HASH 计算

1.2.3.1 工作流程

- 步骤 1 调用 uapi_drv_cipher_hash_start 接口创建一路 hash，配置当前 hash 计算类型，并获取 hash 句柄。
- 步骤 2 调用 uapi_drv_cipher_hash_update 接口传入消息数据，对消息进行摘要计算。
- 步骤 3 调用 uapi_drv_cipher_hash_final 接口获取消息摘要。

----结束

说明

步骤 3 如果成功会自己销毁 hash 句柄，不需要额外调用 hash 接口。

1.2.3.2 代码示例

```
uint32_t g_test_buf[] = {
    0xF947C225, 0xD6F52F9A, 0xBE3DED98, 0xB48EDE4D, 0xE1303291, 0x31B23930,
    0xBF0CCD3D, 0x824F17A8,
    0x17FBE151, 0x60E3EAAE, 0xCDBF0CDA, 0x1E7508C7, 0x7ED42484, 0xEA482E69,
    0x5F61D406, 0x1CB7D87B,
    0xC91DACD2, 0x2CECD543, 0x5C58E135, 0x0257978B, 0x3E102E6A, 0x2B6CFF22,
    0x51B070BB, 0x3DC93AE8,
};

errcode_t hash_sha256_test(void)
{
    errcode_t ret;
    uint32_t handle;
    uint32_t check_sum;
    uint32_t result_len;
    uint32_t block_size = 64;
    drv_cipher_hash_attr_t hash_attr;
    hash_attr.hash_type = DRV_CIPHER_HASH_TYPE_SHA256;
    drv_cipher_buf_attr_t buf;
    buf.address = (uint8_t*)(uint32_t)g_test_buf;
```

```
check_sum = (uint32_t)(&handle) ^ (uint32_t)(&hash_attr);
ret = uapi_drv_cipher_hash_start(&handle, &hash_attr, check_sum);
if (ret != ERRCODE_SUCC) {
    return ret;
}

// 计算长度为65字节
int32_t update_len = block_size + 1;
check_sum = (uint32_t)handle ^ (uint32_t)(&buf) ^ update_len;
ret = uapi_drv_cipher_hash_update(handle, &buf, update_len, 0, check_sum);
if (ret != ERRCODE_SUCC) {
    return ret;
}

check_sum = (uint32_t)handle ^ (uint32_t)g_hash_result ^ (uint32_t)(&result_len);
ret = uapi_drv_cipher_hash_final(handle, g_hash_result, &result_len, check_sum);
if (ret != ERRCODE_SUCC) {
    return ret;
}

return ERRCODE_SUCC;
}
```

1.2.3.3 注意事项

- 在调用 uapi_drv_cipher_hash_final 接口获取消息摘要时，传入的存储结果的 buffer 需要调用者申请，其大小为 out_len，且 out_len 不应小于当前 hash 类型对应的摘要计算结果长度。
- 支持多段数据连续更新，结果与多段数据一次性更新结果一致。
- 支持重试，即在调用 uapi_drv_cipher_hash_update/uapi_drv_cipher_hash_final 接口失败时，仍可配置正确参数后继续重新调用该接口，完成后续摘要计算操作。

1.2.4 RSA 加解密

1.2.4.1 工作流程

步骤 1 公钥加密，调用 uapi_drv_cipher_pke_rsa_public_encrypt 接口完成。

步骤 2 私钥解密，调用 uapi_drv_cipher_pke_rsa_private_decrypt 接口完成。

----结束

说明

调用加密接口时，存储加密结果的 buffer，其缓冲区大小不能小于密钥 N 的位宽。（RSA 算法中，公钥为 N,E，私钥为 N,D）

1.2.4.2 代码示例

```
static uint8_t common_rsa4096_n[] = {  
    0xdd, 0xaa, 0x93, 0xb7, 0xee, 0x7f, 0x0c, 0x8d, 0x77, 0x9b, 0x84, 0x03, 0x34, 0x11, 0x93,  
    0x9b,  
    0x58, 0x9e, 0x7b, 0xc9, 0x50, 0xb0, 0xc4, 0xfb, 0x11, 0x6f, 0xc0, 0xf6, 0x1c, 0x4a, 0x17, 0xc3,  
    0xb2, 0x98, 0x4b, 0xb4, 0x95, 0xae, 0x04, 0x07, 0x60, 0xd1, 0xd8, 0x66, 0xd2, 0x9b, 0x1e,  
    0x31,  
    0x2a, 0x5e, 0xe5, 0x50, 0xce, 0x13, 0xd5, 0x84, 0x40, 0x43, 0x3b, 0x6c, 0x64, 0x72, 0x68,  
    0x07,  
    0x73, 0x35, 0x57, 0x17, 0x60, 0xcd, 0x3a, 0xd1, 0x6a, 0x5a, 0x24, 0x67, 0xde, 0x12, 0x65,  
    0x9d,  
    0xf6, 0x6a, 0x8c, 0x7c, 0x57, 0xbd, 0x69, 0x15, 0xdb, 0xbb, 0xb0, 0xff, 0xd1, 0xfe, 0xf3, 0x81,  
    0xb6, 0x22, 0x7d, 0xcc, 0xde, 0x24, 0x59, 0xcd, 0xca, 0x5c, 0x07, 0xc9, 0x77, 0xa5, 0xd1,  
    0x3e,  
    0xc5, 0xda, 0xc9, 0xdf, 0x92, 0x33, 0x27, 0x96, 0x50, 0x50, 0x1a, 0x3e, 0xde, 0x11, 0xf2, 0x0f,  
    0xca, 0x04, 0x01, 0xdb, 0x58, 0x60, 0x6e, 0x36, 0x9b, 0xd1, 0x2a, 0xed, 0x2a, 0xac, 0x96,  
    0x49,  
    0x97, 0x45, 0x1b, 0x2d, 0x44, 0xc1, 0xbc, 0x27, 0x04, 0xd5, 0xd7, 0x8e, 0xfc, 0xf4, 0x50,  
    0x34,  
    0x2c, 0x5c, 0x50, 0xa8, 0xfb, 0x66, 0xcd, 0xe8, 0x77, 0x14, 0x36, 0xcb, 0xf6, 0x63, 0xdc,  
    0xd4,  
    0xa9, 0xd1, 0xdf, 0xbd, 0xd0, 0xc2, 0x6e, 0xd3, 0xe9, 0x3c, 0x29, 0x3d, 0xa5, 0x6b, 0x5a,  
    0x2a,  
    0xed, 0x16, 0xa8, 0x45, 0xcf, 0xaa, 0xc2, 0x75, 0xb1, 0x98, 0xa9, 0x14, 0xf9, 0xd9, 0xf1, 0xb7,  
    0x95, 0x13, 0x0b, 0xb3, 0x4b, 0xee, 0xcd, 0xcb, 0xb8, 0xb9, 0x59, 0x33, 0xb9, 0x64, 0x5b,  
    0xad,  
    0x1d, 0xd3, 0x90, 0x4b, 0x1e, 0xba, 0x41, 0x15, 0x29, 0xd1, 0x46, 0xa7, 0x5d, 0xfe, 0x96,  
    0x3f,  
    0x91, 0x29, 0xe1, 0x30, 0xf2, 0xeb, 0xde, 0xd9, 0xd4, 0x75, 0xfe, 0x7b, 0x4e, 0x93, 0xe0,  
    0xd6,  
    0x7a, 0x85, 0x8d, 0xbf, 0xaa, 0x7b, 0x38, 0x79, 0xec, 0xe3, 0xdd, 0x1f, 0xc2, 0x55, 0x97,  
    0x20,  
    0x20, 0x92, 0x48, 0x21, 0x02, 0xce, 0x69, 0x04, 0x26, 0x1b, 0x3b, 0x1b, 0x7f, 0xd0, 0xd8,  
    0x19,  
    0xa8, 0x6a, 0x30, 0xa8, 0x4b, 0xae, 0xfe, 0xfc, 0x86, 0x0a, 0x45, 0xb6, 0x80, 0x50, 0x46,  
    0xda,  
    0xf5, 0x29, 0x4c, 0xce, 0x89, 0xc6, 0x77, 0x87, 0x7d, 0xa7, 0xe2, 0x8e, 0x0e, 0x5b, 0xed,  
    0x5d,
```

```
0xf5, 0xc7, 0xcd, 0xf2, 0x2e, 0x33, 0xf3, 0x64, 0x98, 0xd1, 0xea, 0x55, 0x78, 0x5a, 0xb3, 0xc0,
0xea, 0xda, 0xe8, 0xf2, 0xf1, 0x2b, 0x0c, 0x82, 0x5f, 0x1a, 0xf1, 0x0d, 0x6b, 0x36, 0x7f, 0x34,
0xe1, 0xd8, 0x73, 0xd7, 0x74, 0x55, 0xaa, 0x06, 0x86, 0x15, 0xf8, 0xe0, 0x01, 0x32, 0x65,
0x53,
0x82, 0x19, 0xb1, 0x3d, 0xb6, 0x93, 0xd1, 0xdc, 0x35, 0x8b, 0xe9, 0x03, 0x90, 0xa1, 0x9b,
0x25,
0x30, 0x58, 0xdc, 0x6b, 0xe2, 0x8a, 0x43, 0xc8, 0x92, 0x1d, 0x64, 0xa6, 0x8d, 0xa9, 0x00,
0xfa,
0x0c, 0x24, 0x7e, 0xe9, 0xd7, 0x06, 0xd1, 0xbe, 0x99, 0x63, 0xe0, 0x85, 0x3f, 0x79, 0x41,
0xb6,
0x46, 0x43, 0x98, 0x14, 0x3c, 0x89, 0x6a, 0x63, 0x54, 0x40, 0x85, 0xe1, 0xda, 0x21, 0x03,
0xf3,
0xb3, 0x87, 0x18, 0x0d, 0x3c, 0x29, 0xd8, 0x96, 0x6f, 0xf6, 0x67, 0x55, 0x4b, 0x24, 0x44,
0xd3,
0x14, 0xdb, 0x47, 0xb7, 0x49, 0x09, 0x81, 0x99, 0x6e, 0x0e, 0x03, 0xfc, 0xc8, 0x72, 0x35,
0x67,
0xfe, 0x7b, 0x5e, 0x77, 0xfe, 0x5f, 0x71, 0xd9, 0xb0, 0xb6, 0xa6, 0x09, 0xa6, 0x71, 0xbe,
0x2d,
0x3c, 0xaa, 0x96, 0x31, 0xdf, 0x3f, 0x26, 0x4c, 0xed, 0x8f, 0x71, 0xa1, 0xb4, 0x70, 0xfd, 0xad,
0x95, 0x5d, 0xf4, 0xc3, 0xef, 0x89, 0x62, 0xd9, 0xc5, 0x9a, 0xad, 0x0d, 0x04, 0x32, 0x4f, 0x33,
};

static uint8_t common_rsa4096_d[] = {
0xb5, 0x88, 0x27, 0x57, 0x5f, 0x5a, 0xee, 0xc5, 0xc0, 0x29, 0x3d, 0x10, 0x7e, 0x88, 0xd2,
0x70,
0x4b, 0x3f, 0xe7, 0x32, 0x34, 0x01, 0xc0, 0x1f, 0xb8, 0xe4, 0xe3, 0x8a, 0xea, 0x1a, 0x07,
0xa2,
0x3d, 0xd5, 0x99, 0x52, 0x37, 0xae, 0x7e, 0x20, 0x28, 0xbb, 0x51, 0xd4, 0xcb, 0x2f, 0x3b,
0xa7,
0x9a, 0x02, 0x83, 0x1c, 0x0c, 0xd8, 0x93, 0x68, 0xae, 0x54, 0x21, 0x0b, 0x20, 0xab, 0xcc,
0xe4,
0x25, 0x06, 0x8e, 0xdf, 0x57, 0x68, 0x5b, 0x7d, 0xfa, 0xf1, 0xfd, 0x94, 0x8e, 0x7a, 0x54, 0x7b,
0xeb, 0xbc, 0xd0, 0x76, 0x58, 0x48, 0x87, 0x11, 0xde, 0x94, 0xb4, 0x5c, 0x9c, 0xf6, 0x85,
0x27,
0x3a, 0x28, 0xbf, 0x0b, 0x92, 0xf5, 0x04, 0x12, 0x93, 0x61, 0x91, 0x02, 0xfe, 0x18, 0x6e,
0xe7,
0x50, 0x93, 0x5f, 0xf5, 0xd7, 0x3e, 0x4b, 0x72, 0x3f, 0x2d, 0x8a, 0x80, 0xe7, 0xce, 0x9c, 0x85,
0x2f, 0xb4, 0xde, 0x6c, 0x6a, 0xd0, 0xf6, 0x11, 0x84, 0xc3, 0xe4, 0xba, 0xbb, 0xd3, 0x01,
0x75,
0x1d, 0x0b, 0xfc, 0x38, 0xb3, 0x71, 0x51, 0x8c, 0x46, 0xda, 0x75, 0xa0, 0xe5, 0x29, 0x93,
0xb1,
0x56, 0x8e, 0xf7, 0x83, 0x9b, 0xf7, 0x52, 0x33, 0xc9, 0xa9, 0x65, 0x42, 0xdd, 0xf2, 0x64, 0x7c,
0x48, 0xe2, 0xd6, 0xb0, 0x15, 0x91, 0xd5, 0xbf, 0x77, 0xe7, 0xcc, 0x02, 0x6f, 0x41, 0x1e,
```



```

0x63,
    0xbf, 0x2c, 0x69, 0xfc, 0x5a, 0x18, 0x87, 0x0e, 0x69, 0xb6, 0x12, 0xea, 0x59, 0xbf, 0x91, 0xc3,
    0xfd, 0xb2, 0xce, 0x47, 0x34, 0xad, 0x4a, 0x1e, 0x47, 0x96, 0x8b, 0x25, 0xdf, 0xf2, 0xff, 0x5d,
    0x23, 0xeb, 0x09, 0xe5, 0x6b, 0x31, 0xaf, 0x71, 0x0c, 0x81, 0x15, 0xb2, 0xa5, 0x38, 0x84,
0x85,
    0x64, 0x75, 0x7e, 0xb5, 0x5e, 0x8b, 0xaa, 0x42, 0x8c, 0x4b, 0x61, 0x70, 0x4c, 0x26, 0xb0,
0xde,
    0x8e, 0x3a, 0x14, 0xee, 0xda, 0x46, 0x8d, 0xeb, 0x52, 0xa0, 0xc3, 0xe0, 0x9a, 0x54, 0xa0,
0x38,
    0x04, 0xc1, 0x34, 0x6c, 0x39, 0x80, 0x83, 0x48, 0xb9, 0x83, 0xf5, 0xbe, 0xa3, 0x07, 0x77,
0x8d,
    0xf2, 0xa8, 0x13, 0xda, 0xf1, 0xdd, 0x34, 0x24, 0x3e, 0xd9, 0xb4, 0xb9, 0x56, 0xb1, 0xfb,
0xa0,
    0x2b, 0x41, 0x08, 0x57, 0x01, 0x3b, 0x22, 0x0b, 0x5a, 0x15, 0x53, 0xf7, 0x6e, 0xda, 0x49,
0x42,
    0xd3, 0x77, 0x2a, 0x8d, 0xed, 0x8e, 0x5d, 0x89, 0xf8, 0xd3, 0x5b, 0xaf, 0x61, 0xd2, 0x2d,
0x92,
    0x14, 0xf2, 0x52, 0xb4, 0xa7, 0x8a, 0xab, 0x01, 0xde, 0xee, 0xee, 0xd5, 0x14, 0xb1, 0x27,
0x5b,
    0xb9, 0x8a, 0x90, 0xb2, 0x4a, 0x73, 0xb2, 0x48, 0x9c, 0x36, 0x6c, 0xd5, 0x96, 0xfe, 0xc6,
0xa2,
    0x0c, 0xa8, 0x61, 0xc8, 0xa0, 0x3b, 0xf0, 0xab, 0xf8, 0xac, 0x41, 0x38, 0xbf, 0xb2, 0xae, 0x95,
    0xf3, 0x51, 0xb9, 0x37, 0x3e, 0xd4, 0x81, 0x4c, 0x1a, 0xae, 0x5a, 0x2e, 0xb4, 0x5d, 0x14,
0x80,
    0x36, 0x03, 0x9c, 0x72, 0xc4, 0xe7, 0x26, 0x0e, 0x42, 0x95, 0xf6, 0xd2, 0xa4, 0x3a, 0x5c,
0xad,
    0x5f, 0x5e, 0xf0, 0x39, 0x18, 0x75, 0x45, 0x55, 0x3d, 0xfd, 0x12, 0xda, 0x05, 0xe4, 0x6c,
0x73,
    0x33, 0xc6, 0x09, 0x83, 0x0f, 0x56, 0x23, 0x06, 0x78, 0x0a, 0x76, 0x3a, 0x30, 0x31, 0x56,
0x90,
    0xfb, 0x7f, 0x17, 0xaf, 0x65, 0xa4, 0x39, 0x09, 0x47, 0x15, 0x9c, 0x30, 0xea, 0x89, 0xdf, 0xaf,
    0x18, 0x97, 0xd1, 0xf9, 0xad, 0x59, 0xaa, 0x94, 0xd2, 0x83, 0xa7, 0x92, 0x82, 0xa1, 0xab,
0x82,
    0xd7, 0x9d, 0x68, 0x32, 0xf3, 0x92, 0x76, 0xb0, 0x68, 0x1a, 0xe0, 0x0f, 0x92, 0x6e, 0xd7,
0x8f,
    0x03, 0x78, 0x8e, 0xbe, 0xfb, 0x52, 0x38, 0xa0, 0x93, 0xeb, 0xb4, 0x41, 0xf1, 0xeb, 0x5b,
0xd9,
};

static uint8_t common_rsa4096_e[] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
    0x00,
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

```

[illegible]

20

```

uint32_t test_data_len = sizeof(rsa_msg);
drv_pke_rsa_scheme_t scheme = DRV_PKE_RSA_SCHEME_PKCS1_V21;
drv_pke_rsa_priv_key_t priv_key = {.n = (uint8_t *)common_rsa4096_n, .d = (uint8_t
*)common_rsa4096_d, .n_len = sizeof(common_rsa4096_n), .d_len = sizeof(common_rsa4096_d)};
drv_pke_rsa_pub_key_t pub_key = {.n = (uint8_t *)common_rsa4096_n, .e = (uint8_t
*)common_rsa4096_e, .len = sizeof(common_rsa4096_n)};

input.data = test_data;
input.length = test_data_len;
label.data = label_data;
label.length = sizeof(label_data);
output.data = enc_data;
output.length = DRV_PKE_LEN_4096;

check_wd = (uint32_t)scheme ^ DRV_PKE_HASH_TYPE_SHA256 ^ (uint32_t)&pub_key ^
(uint32_t)&input ^
(uint32_t)&label ^ (uint32_t)&output;
ret = uapi_drv_cipher_pke_rsa_public_encrypt(scheme, DRV_PKE_HASH_TYPE_SHA256,
&pub_key, &input, &label, &output, check_wd);
if (ret != ERRCODE_SUCC) {
    PRINT("uapi_drv_cipher_pke_rsa_public_encrypt failed\n");
    return ret;
}

input.data = enc_data;
input.length = DRV_PKE_LEN_4096;
output.data = dec_data;
output.length = DRV_PKE_LEN_4096;
check_wd = (uint32_t)scheme ^ DRV_PKE_HASH_TYPE_SHA256 ^ (uint32_t)&priv_key ^
(uint32_t)&input ^
(uint32_t)&label ^ (uint32_t)&output;
ret = uapi_drv_cipher_pke_rsa_private_decrypt(scheme, DRV_PKE_HASH_TYPE_SHA256,
&priv_key, &input, &label, &output, check_wd);
if (ret != ERRCODE_SUCC) {
    PRINT("uapi_drv_cipher_pke_rsa_private_decrypt failed\n");
    return ret;
}

if (test_data_len != output.length) {
    PRINT("compare output length failed\n");
    return ret;
} else if (output.length != 0) {
    ret = memcmp(dec_data, test_data, test_data_len);

```

```
if (ret != ERRCODE_SUCC) {  
    PRINT("memcmp failed\n");  
}  
}  
  
return ERRCODE_SUCC;  
}
```

1.2.4.3 注意事项

支持 PKCS#1 v1.5 和 PKCS#1 v2.1 OAEP 两种模式，且用户标签数据（label）可选，该 label 仅在 PKCS#1 v2.1 OAEP 模式下使用。

须知

PKCS#1 v1.5 填充方式不安全，不建议使用。

1.2.5 RSA 签名验签

1.2.5.1 工作流程

步骤 1 私钥签名，调用 uapi_drv_cipher_pke_rsa_sign 接口完成。

步骤 2 公钥验签，调用 uapi_drv_cipher_pke_rsa_verify 接口完成。

----结束

1.2.5.2 代码示例

// 以下示例中使用的key与1.2.4.2小节中的相同。

```
errcode_t rsa_pss_sign_test(void)  
{  
    errcode_t ret;  
    uint32_t check_wd;  
    drv_pke_data_t input_hash, output_hash, sign;  
    uint8_t sign_data[DRV_PKE_LEN_4096];  
    uint8_t hash_data[SHA256_RESULT_LENGTH];  
    uint8_t *hash = hash_sha256;  
    drv_pke_rsa_scheme_t scheme = DRV_PKE_RSA_SCHEME_PKCS1_V21;  
    drv_pke_rsa_priv_key_t priv_key = {.n = (uint8_t *)common_rsa4096_n, .d = (uint8_t *)common_rsa4096_d, .n_len = sizeof(common_rsa4096_n), .d_len = sizeof(common_rsa4096_d)};  
    drv_pke_rsa_pub_key_t pub_key = {.n = (uint8_t *)common_rsa4096_n, .e = (uint8_t
```

```

*)common_rsa4096_e, .len = sizeof(common_rsa4096_n));

    input_hash.data = hash;
    input_hash.length = SHA256_RESULT_LENGTH;
    output_hash.data = hash_data;
    output_hash.length = SHA256_RESULT_LENGTH;
    sign.data = sign_data;
    sign.length = pub_key.len;

    // 签名接口

    check_wd = (uint32_t)&priv_key ^ (uint32_t)scheme ^ DRV_PKE_HASH_TYPE_SHA256 ^
                (uint32_t)&input_hash ^ (uint32_t)&sign;
    ret = uapi_drv_cipher_pke_rsa_sign(&priv_key, scheme, DRV_PKE_HASH_TYPE_SHA256,
    &input_hash, &sign, check_wd);
    if (ret != ERRCODE_SUCC) {
        PRINT("uapi_drv_cipher_pke_rsa_sign failed\n");
        return ret;
    }

    // 验签接口

    check_wd = (uint32_t)&pub_key ^ (uint32_t)scheme ^ DRV_PKE_HASH_TYPE_SHA256 ^
                (uint32_t)&input_hash ^
                (uint32_t)&sign ^ (uint32_t)&output_hash;
    ret = uapi_drv_cipher_pke_rsa_verify(&pub_key, scheme, DRV_PKE_HASH_TYPE_SHA256,
    &input_hash, &sign, &output_hash, check_wd);
    if (ret != ERRCODE_SUCC) {
        PRINT("uapi_drv_cipher_pke_rsa_verify failed\n");
        return ret;
    }

    ret = memcmp(hash_data, input_hash.data, input_hash.length);
    if (ret != ERRCODE_SUCC) {
        PRINT("memcmp hash after verify failed\n");
        return ret;
    }
    return ERRCODE_SUCC;
}

```

1.2.5.3 注意事项

支持 PKCS#1 v1.5 和 PKCS#1 v2.1 PSS 两种模式的私钥签名和公钥验签。

须知

PKCS#1 v1.5 填充方式不安全，不建议使用。

1.2.6 大数模幂运算

1.2.6.1 工作流程

调用 `uapi_drv_cipher_pke_exp_mod` 接口获取大数模幂运算结果。

1.2.6.2 代码示例

```
uint32_t g_key[] = {  
    // N  
    0x855021B5, 0x274926E5, 0xB3A06D25, 0x460D4043, 0xED5064AE, 0xCA5CEDF4,  
    0x2BAEBD57, 0xF045ED39,  
    0xB5971F36, 0x61A103CB, 0x8E0CD2FC, 0x314D6E60, 0xA3F95F45, 0xEB4A8407,  
    0x17A2529B, 0xE9411EF8,  
    0x5FA172E1, 0xB4D6B569, 0xD4E27F74, 0x4797813C, 0x95A4DE44, 0xFB476337,  
    0xE742C044, 0xA370718F,  
    0x427F83A5, 0xA937BDE0, 0x80A33F7A, 0x49091282, 0xE30FEE60, 0xF06605DA,  
    0x71FF1305, 0x32C84ABE,  
    0xC456416C, 0xFBEF34EF, 0xB58D9DBD, 0xD397627C, 0xA18273DB, 0xD8F8FC0A,  
    0xB721E1EF, 0x126F3C54,  
    0x3267F13F, 0x906F98CF, 0xBF982901, 0xADEF92D4, 0xF873EA4C, 0x5B44333B,  
    0xF039188A, 0x9FFB5F47,  
    0x3AC8186C, 0xCD939F41, 0x070782A5, 0xF21B91C8, 0x55252632, 0x1EA7F882,  
    0x7686851A, 0x85BCE81B,  
    0x94A0CC9C, 0x7F02FFD8, 0x511C1DE6, 0xF36A70C1, 0x4FC88628, 0xCD7C04A2,  
    0xF1EDF23B, 0xD1AA4AA5,  
    0x9C595F2E, 0xB808A11F, 0x42C37C63, 0xCB852E94, 0x57814A10, 0x4E5FAE20,  
    0xDD5FBB95, 0xFC5EFBC4,  
    0x133FECFE, 0xA0284C23, 0xFBB613A1, 0x6BCE8A07, 0x069B5362, 0x62BF4987,  
    0x4B5322A2, 0xD59AE38D,  
    0xDE729014, 0x172402F8, 0xBFEAE887, 0x05D2CF72, 0x1BE804BE, 0x1901A2F7,  
    0x1AFF04E3, 0x03FC4A6E,  
    0xB584F0C7, 0x72A438E9, 0x455B9137, 0x19427F45, 0x7BFE5B08, 0x2ABA6C49,  
    0x06A8B4AD, 0xDF99CCF3,  
    0x6B7ACC12, 0x8A413242, 0x2FC38F51, 0x01A40687, 0xE5D64A5D, 0xC32A48F6,  
    0x482F6768, 0x264BA9D0,  
    0x87B2B2BF, 0x65589620, 0xB3661DA5, 0xCC141C44, 0x42C4D9CC, 0x54700176,  
    0x457513F8, 0x74944A48,  
    0xF03AAC38, 0x8AC0E391, 0xFC8DF177, 0x78B799A2, 0x58E11835, 0x9A8601B4,
```

[illegible]


```

    0xA7F9FA0F, 0x84831462, 0xC2E676FA, 0xC9ED1C2F, 0x8E5882E1, 0xE6DF87E8,
    0xC40AC368, 0x4C7CF2F6,
    0xDDE2F33F, 0xBE7EBF73, 0xADB59913, 0xA1B5C6DB, 0x71D9E2EB, 0x8226FB34,
    0x88124956, 0xC36F0079,
    0xB66720DD, 0xB6AB81C6, 0xFD86D178, 0x3132F297, 0xB6AF98A7, 0xD91A7240,
    0xF5E54FB8, 0x512A711E,
    0x33A4E755, 0x14EB5834, 0x48AB675B, 0xEE5E5BA8, 0xEBCAD1E2, 0x01373A76,
    0x5EFDD78C, 0x891FD47C,
    0xB35BB265, 0xD78F9009, 0x635E2B53, 0x60C7856A, 0x70C05937, 0xE68DB57A,
    0x6EC42EEB, 0x5CEBE19C,
    0xC16108FF, 0x733612D3, 0xD6E9015B, 0x2A0A3EB8, 0xAA472AD4, 0x010D1AC9,
    0xCD13DB46, 0xB0C37944,
};

errcode_t rsa_exp_mod_test(void)
{
    errcode_t ret;
    uint32_t check_sum;
    uint32_t test_result[RSA_4096_RESULT_LENGTH] = { 0x00 };

    drv_pke_data_t n = {
        .data = (uint8_t*)(uint32_t)g_key,
        .length = DRV_PKE_LEN_4096
    };

    drv_pke_data_t k = {
        .data = ((uint8_t*)(uint32_t)g_key + DRV_PKE_LEN_4096),
        .length = DRV_PKE_LEN_4096
    };

    drv_pke_data_t in = {
        .data = (uint8_t*)g_plain_text,
        .length = DRV_PKE_LEN_4096
    };

    drv_pke_data_t out = {
        .data = (uint8_t*)(uint32_t)test_result,
        .length = DRV_PKE_LEN_4096
    };

    check_sum = (uint32_t)&n ^ (uint32_t)&k ^ (uint32_t)&in ^ (uint32_t)&out;
    ret = uapi_drv_cipher_pke_exp_mod(&n, &k, &in, &out, check_sum);
    if (ret != ERRCODE_SUCC) {

```

```
        return ret;
    }

    return ERRCODE_SUCC;
}
```

1.2.6.3 注意事项

接口中涉及的 n、k、in、out 长度需要相等。

1.2.7 TRNG

1.2.7.1 工作流程

调用 `uapi_drv_cipher_trng_get_random` 接口获取随机数。该接口获取的随机数长度为 4Byte，若要获取其他长度的随机数，需要循环调用该接口，请参见“[1.2.7.2 代码示例](#)”。

1.2.7.2 代码示例

```
#define RANDOM_LENGTH 512
#define WORD_SIZE 4
errcode_t get_random(uint8_t *rand, const uint32_t size)
{
    errcode_t ret;
    uint32_t k = 0;
    uint32_t randnum = 0;
    uint8_t randlist[RANDOM_LENGTH] = {0};

    for (k = 0; k < (size + WORD_SIZE - 1) / WORD_SIZE; k++) {
        ret = uapi_drv_cipher_trng_get_random((uint32_t *) (randlist + k * WORD_SIZE));
        if (ret != ERRCODE_SUCC) {
            return ret;
        }
    }

    if (memcpy_s(rand, size, randlist, size) != EOK) {
        return ERRCODE_MEMCPY;
    }

    return ERRCODE_SUCC;
}
```

2 API 参考

- 2.1 KM API
- 2.2 CIPHER API
- 2.3 HASH API
- 2.4 RSA API
- 2.5 TRNG API
- 2.6 错误码

2.1 KM API

2.1.1 uapi_drv_klad_create

【描述】

创建 klad 句柄。

【语法】

```
errcode_t uapi_drv_klad_create(uint32_t *klad_handle);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输出。

【返回值】

返回值	描述
-----	----

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无

2.1.2 uapi_drv_klad_destroy

【描述】

销毁 klad 句柄。

【语法】

```
errcode_t uapi_drv_klad_destroy(uint32_t klad_handle);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- klad 句柄必须已创建。
- 执行 destroy 操作销毁 klad 句柄的 task 必须与创建 klad 句柄的 task 为同一个。

2.1.3 uapi_drv_klad_attach

【描述】

绑定 klad 句柄与 keyslot 通道。

【语法】

```
errcode_t uapi_drv_klad_attach(uint32_t klad_handle, uapi_drv_klad_dest_t klad_type, uint32_t keyslot_handle);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输入。
klad_type	绑定的 keyslot 类型。	输入。
keyslot_handle	当 dest 为 mcipher 或 hmac 时，指 keyslot 句柄；当 dest 为 flash，指 klad_flash_key_type。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

【注意】

- klad 句柄必须已创建。
- 执行 attach 操作的 task 必须与创建 klad 句柄的 task 为同一个。

2.1.4 uapi_drv_klad_detach

【描述】

解绑 klad 句柄与 keyslot 通道。

【语法】

```
errcode_t uapi_drv_klad_detach(uint32_t klad_handle, uapi_drv_klad_dest_t klad_type, uint32_t keyslot_handle);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输入。
klad_type	解绑的 keyslot 类型。	输入。
keyslot_handle	当 dest 为 mcipher 或 hmac 时，指 keyslot 句柄；当 dest 为 flash，指 klad_flash_key_type。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- klad 句柄必须已创建。
- klad 句柄必须已绑定 keyslot 通道。
- 执行 detach 操作的 task 必须与创建 klad 句柄的 task 为同一个。

2.1.5 uapi_drv_klad_set_attr

【描述】

设置 klad 句柄属性。

【语法】

```
errcode_t uapi_drv_klad_set_attr(uint32_t klad_handle, const uapi_drv_klad_attr_t *attr);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输入。
attr	klad 句柄属性指针。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- klad 句柄必须已创建。
- klad 句柄属性指针不能为空。
- 执行 set 操作的 task 必须与创建 klad 句柄的 task 为同一个。

2.1.6 uapi_drv_klad_get_attr

【描述】

获取 klad 句柄属性。

【语法】

```
errcode_t uapi_drv_klad_get_attr(uint32_t klad_handle, uapi_drv_klad_attr_t *attr);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输入。
attr	存储 klad 句柄属性的指针。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- klad 句柄必须已创建。
- klad 句柄属性指针不能为空。
- 执行 get 操作的 task 必须与创建 klad 句柄的 task 为同一个。

2.1.7 uapi_drv_klad_set_clear_key

【描述】

设置明文密钥。

【语法】

```
errcode_t uapi_drv_klad_set_clear_key(uint32_t klad_handle, const  
drv_klad_clear_key_t *key);
```

【参数】

参数	描述	输入/输出
klad_handle	klad 句柄。	输入。
key	存储明文 key 属性的指针。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

【注意】

- klad 句柄必须已创建。
- 存储明文 key 属性的指针不能为空。
- 执行 set 操作的 task 必须与创建 klad 句柄的 task 为同一个。

2.1.8 uapi_drv_keyslot_create

【描述】

创建 keyslot 句柄。

【语法】

```
errcode_t uapi_drv_keyslot_create(uint32_t *keyslot_handle,  
uapi_drv_uapi_drv_keyslot_type_t keyslot_type);
```

【参数】

参数	描述	输入/输出
keyslot_handle	keyslot 句柄。	输入。
keyslot_type	keyslot 类型。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无。

2.1.9 uapi_drv_keyslot_destroy

【描述】

销毁 keyslot 句柄。

【语法】

```
errcode_t uapi_drv_keyslot_destroy(uint32_t keyslot_handle);
```

【参数】

参数	描述	输入/输出
keyslot_handle	keyslot 句柄。	输入。

【返回值】

返回值	描述
-----	----

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- keyslot 句柄必须已创建。
- 执行 destroy 操作的 task 必须与创建 keyslot 句柄的 task 为同一个。

2.2 CIPHER API

2.2.1 uapi_drv_cipher_symc_create

【描述】

创建 cipher 虚拟通道句柄。

【语法】

```
errcode_t uapi_drv_cipher_symc_create(uint32_t *symc_handle, const  
uapi_drv_cipher_symc_attr_t *symc_attr);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输出。
symc_attr	cipher 句柄属性指针。	输入。

后续 check_word 参数含义与此处一致，不再一一做说明。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无。

2.2.2 uapi_drv_cipher_symc_destroy

【描述】

销毁 cipher 虚拟通道句柄。

【语法】

```
errcode_t uapi_drv_cipher_symc_destroy(uint32_t symc_handle);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

【注意】

- cipher 句柄必须已创建。
- 执行销毁操作前必须已经执行了 detach 操作解绑 cipher 句柄与 keyslot 通道。
- 执行 destroy 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.3 uapi_drv_cipher_symc_set_config

【描述】

配置 cipher 句柄属性。

【语法】

```
errcode_t uapi_drv_cipher_symc_set_config(uint32_t symc_handle, const  
uapi_drv_cipher_symc_ctrl_t *symc_ctrl);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
symc_ctrl	cipher 密码算法属性配置指针，例如算法、工作模式、iv、密钥长度、密钥奇偶属性等。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- cipher 句柄必须已创建。
- AES-GCM 算法，必须传入 iv。
- 支持的 IV 长度和 Tag 长度均为 128bit。
- 执行 set 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.4 uapi_drv_cipher_symc_get_config

【描述】

获取 cipher 虚拟通道句柄属性。

【语法】

```
errcode_t uapi_drv_cipher_symc_get_config(uint32_t symc_handle,  
uapi_drv_cipher_symc_ctrl_t *symc_ctrl);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
symc_ctrl	cipher 密码算法属性配置指针，例如算法、工作模式、iv、密钥等。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- cipher 句柄必须已创建。
- cipher 句柄属性必须已配置。
- 执行 get 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.5 uapi_drv_cipher_symc_attach

【描述】

绑定 cipher 句柄与 keyslot 通道。

【语法】

```
errcode_t uapi_drv_cipher_symc_attach(uint32_t symc_handle, uint32_t  
keyslot_handle);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
keyslot_handle	keyslot 通道。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- cipher 句柄必须已创建。
- attach 的 keyslot 通道类型必须为 KEYSLOT_TYPE_MCIPHER。
- 执行 attach 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.6 uapi_drv_cipher_symc_detach

【描述】

解绑 cipher 句柄与 keyslot 通道。

【语法】

```
errcode_t uapi_drv_cipher_symc_detach(uint32_t symc_handle, uint32_t  
keyslot_handle);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
keyslot_handle	keyslot 通道。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

【注意】

- cipher 句柄必须已创建。
- attach 的 keyslot 通道类型必须为 KEYSLOT_TYPE_MCIPHER。
- 执行 attach 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.7 uapi_drv_cipher_symc_encrypt

【描述】

对称加密。

【语法】

```
errcode_t uapi_drv_cipher_symc_encrypt(uint32_t symc_handle, const
uapi_drv_cipher_buf_attr_t *src_buf, const uapi_drv_cipher_buf_attr_t *dst_buf,
uint32_t length);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
src_buf	加密数据源缓冲区属性，包括缓冲区地址与缓冲区安全类型。	输入。
dest_buf	加密结果缓冲区属性，包括缓冲区地址与缓冲区安全类型。	输出。
length	加密数据长度，必须 block_size 对齐。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- cipher 句柄必须已创建。
- cipher 句柄属性必须已配置。
- 必须已经调用 KM 模块将密钥送到了 keyslot 通道。
- 对于 ECB/CBC/CTR 模式，加密数据长度必须 block_size 对齐，且长度不能为 0，AESblock_size 为 16Byte。
- 对于 CCM/GCM 模式，如果已经调用了 get_tag 接口获取 tag 值，则不能再调用该接口。
- 执行 encrypt 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.8 uapi_drv_cipher_symc_decrypt

【描述】

对称解密。

【语法】

```
errcode_t uapi_drv_cipher_symc_decrypt(uint32_t symc_handle, const
uapi_drv_cipher_buf_attr_t *src_buf, const uapi_drv_cipher_buf_attr_t *dst_buf,
uint32_t length);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
src_buf	加密数据源缓冲区属性，包括缓冲区地址与缓冲区安全类型。	输入。
dest_buf	加密结果缓冲区属性，包括缓冲区地址与缓冲区安全类型。	输出。
length	加密数据长度，必须 block_size 对齐。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- cipher 句柄必须已创建。
- cipher 句柄属性必须已配置。
- 必须已经调用 km 模块将密钥送到了 keyslot 通道。
- 对于 ECB/CBC/CTR 模式，加密数据长度必须 block_size 对齐，AESblock_size 为 16Byte。

- 对于 CCM/GCM 模式，如果已经调用了 get_tag 接口获取 tag 值，则不能再调用该接口。
- 执行 decrypt 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.2.9 uapi_drv_cipher_symc_get_tag

【描述】

CCM/GCM 获取 tag 值。

【语法】

```
errcode_t uapi_drv_cipher_symc_get_tag(uint32_t symc_handle, uint8_t *tag,  
uint32_t tag_length);
```

【参数】

参数	描述	输入/输出
symc_handle	cipher 句柄。	输入。
tag	存储 tag 值的缓冲区地址。	输出。
tag_length	存储 tag 值的缓冲区大小指针。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

【注意】

- cipher 句柄必须已创建。
- 必须已经调用对称加解密接口完成数据的加解密操作。
- 只有 CCM/GCM 模式可以调用该接口。
- 执行 get_tag 操作的 task 必须与创建 cipher 句柄的 task 为同一个。

2.3 HASH API

2.3.1 uapi_drv_cipher_hash_start

【描述】

创建 hash 句柄。

【语法】

```
errcode_t uapi_drv_cipher_hash_start(uint32_t *hash_handle, const  
uapi_drv_cipher_hash_attr_t *hash_attr);
```

【参数】

参数	描述	输入/输出
hash_handle	hash 句柄。	输出。
hash_attr	配置 hash 句柄基本属性，包括 hash 类型	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

2.3.2 uapi_drv_cipher_hash_update

【描述】

HASH 算法的计算。

【语法】

```
errcode_t uapi_drv_cipher_hash_update (uint32_t hash_handle, const  
uapi_drv_cipher_buf_attr_t *src_buf, const uint32_t len);
```

【参数】

参数	描述	输入/输出
hash_handle	hash 句柄。	输入。

参数	描述	输入/输出
src_buf	源缓冲区属性，包括缓冲区地址与缓冲区安全类型。	输入。
len	缓冲区大小。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“ 2.6 错误码 ”。

【注意】

- hash 句柄必须已创建。
- 如已调用 uapi_drv_cipher_hash_final 接口获取摘要信息，则不能再次进行该计算。
- 该接口支持重试，即若某次过程计算失败，仍可调用该接口重新计算直至成功，而不需要重新开始计算流程。
- 支持连续传入的数据非 block_size 对齐。
- 执行 update 操作的 task 必须与创建 hash 句柄的 task 为同一个。

2.3.3 uapi_drv_cipher_hash_final

【描述】

HASH 获取摘要信息，并在计算成功的时候销毁 hash 句柄。

【语法】

```
errcode_t uapi_drv_cipher_hash_final(uint32_t hash_handle, uint8_t *out, uint32_t *out_len);
```

【参数】

参数	描述	输入/输出
hash_handle	hash 句柄。	输入。
out	存储摘要信息的缓冲区地址指针。	输出。

参数	描述	输入/输出
out_len	存储摘要信息的缓冲区大小指针。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

- hash 句柄必须已创建。
- 该接口支持重试，即若某次过程计算失败，仍可调用该接口重新计算直至成功，而不需要重新开始计算流程。
- 执行 final 操作的 task 必须与创建 hash 句柄的 task 为同一个。

2.4 RSA API

2.4.1 uapi_drv_cipher_pke_rsa_public_encrypt

【描述】

使用 RSA 公钥加密一段明文。

【语法】

```
errcode_t  
uapi_drv_cipher_pke_rsa_public_encrypt(uapi_drv_cipher_pke_rsa_scheme_t  
scheme, uapi_drv_cipher_pke_hash_type_t hash_type, const  
uapi_drv_cipher_pke_rsa_pub_key_t *pub_key, const uapi_drv_cipher_pke_data_t  
*input, const uapi_drv_cipher_pke_data_t *label, uapi_drv_cipher_pke_data_t  
*output);
```

【参数】

参数	描述	输入/输出
scheme	RSA 填充模式，例如： PKCS1_V21/PKCS1_V15。	输入。

参数	描述	输入/输出
hash_type	Hash 类型, PSS-OAEP 模式会对其进行检查。	输入。
pub_key	RSA 公钥。	输入。
input	待加密数据。	输入。
label	用户标签数据结构体, 可选并且只在 OAEP 模式中使用。	输入。
output	加密结果数据, 它的缓冲区大小不能小于 RSA 密钥 N 的位宽。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无

2.4.2 uapi_drv_cipher_pke_rsa_private_decrypt

【描述】

使用 RSA 私钥解密一段密文。

【语法】

```
errcode_t
uapi_drv_cipher_pke_rsa_private_decrypt(uapi_drv_cipher_pke_rsa_scheme_t
scheme, uapi_drv_cipher_pke_hash_type_t hash_type, const
uapi_drv_cipher_pke_rsa_priv_key_t *priv_key, const uapi_drv_cipher_pke_data_t
*input, const uapi_drv_cipher_pke_data_t *label, const uapi_drv_cipher_pke_data_t
*output);
```

【参数】

参数	描述	输入/输出
----	----	-------

参数	描述	输入/输出
scheme	RSA 填充模式。例如： PKCS1_V21/PKCS1_V15。	输入。
hash_type	Hash 类型，PSS-OAEP 模式会对其进行检查。	输入。
priv_key	RSA 私钥。	输入。
input	待解密数据。	输入。
label	用户标签数据结构体，可选并且只在 OAEP 模式中使用。	输入。
output	解密结果数据。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无

2.4.3 uapi_drv_cipher_pke_rsa_sign

【描述】

使用 RSA 私钥签名一段文本。

【语法】

```
errcode_t uapi_drv_cipher_pke_rsa_sign(const uapi_drv_cipher_pke_rsa_priv_key_t
*priv_key, uapi_drv_cipher_pke_rsa_scheme_t scheme,
uapi_drv_cipher_pke_hash_type_t hash_type, const uapi_drv_cipher_pke_data_t
*input_hash, uapi_drv_cipher_pke_data_t *sign);
```

【参数】

参数	描述	输入/输出
priv_key	RSA 私钥。	输入。
scheme	RSA 填充模式, 例如: PKCS1_V21/PKCS1_V15。	输入。
hash_type	Hash 类型。	输入。
input_hash	待签名的摘要。	输入。
sign	签名信息的数据结构体。	输出。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无

2.4.4 uapi_drv_cipher_pke_rsa_verify

【描述】

RSA 公钥验签。

【语法】

```
errcode_t uapi_drv_cipher_pke_rsa_verify(const
uapi_drv_cipher_pke_rsa_pub_key_t *pub_key, uapi_drv_cipher_pke_rsa_scheme_t
scheme, uapi_drv_cipher_pke_hash_type_t hash_type, uapi_drv_cipher_pke_data_t
*input_hash, const uapi_drv_cipher_pke_data_t *sign);
```

【参数】

参数	描述	输入/输出
pub_key	RSA 公钥。	输入。
scheme	RSA 填充模式, 例如:	输入。

参数	描述	输入/输出
	PKCS1_V21/PKCS1_V15。	
hash_type	签名使用的 hash 类型。	输入。
input_hash	待签名消息的 hash 摘要数据。	输入。
sign	签名结果。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

当验签模式为 RSA-PSS 时，入参#input_hash 将会被更改。

2.4.5 uapi_drv_cipher_pke_exp_mod

【描述】

$out = in ^ k \bmod n$ ，模的幂运算。

【语法】

```
errcode_t uapi_drv_cipher_pke_exp_mod(const uapi_drv_cipher_pke_data_t *n,
const uapi_drv_cipher_pke_data_t *k, const uapi_drv_cipher_pke_data_t *in, const
uapi_drv_cipher_pke_data_t *out);
```

【参数】

参数	描述	输入/输出
n	公钥参数 n。	输入
k	公钥参数 e 或者私钥参数 d。	输入
in	输入数据。	输入
out	输出数据。	输出

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

接口中涉及的 n、k、in、out 长度需要相等。

2.5 TRNG API

2.5.1 uapi_drv_cipher_trng_get_random

【描述】

获取硬件随机数。

【语法】

```
errcode_t uapi_drv_cipher_trng_get_random (uint32_t *randnum);
```

【参数】

参数	描述	输入/输出
randnum	指向存储生成的随机数的缓冲区的指针。	输入。

【返回值】

返回值	描述
ERRCODE_SUCC	成功。
非 ERRCODE_SUCC	请参见“2.6 错误码”。

【注意】

无。

2.6 错误码

宏定义		错误码	描述
公共	ERRCODE_SUCC	0	成功。
	ERRCODE_FAIL	0xFFFFFFFF	失败。
	ERRCODE_INVALID_PARAM	0x80000001	参数错误。
	ERRCODE_MEMSET	0x80000003	memset_ss 失败。
	ERRCODE_MEMCPY	0x80000004	memcpy_s 失败。
	ERRCODE_MALLOC	0x80000005	malloc 失败。
km	ERRCODE_KM_INVALID_PARAMETER	0x80001261	参数错误。
	ERRCODE_KM_ENV_NOT_READY	0x80001263	环境未准备好，一般是函数调用顺序出错。
	ERRCODE_KM_KEYSLOT_BUSY	0x80001264	当前使用的 keslot 数量已经超出最大数量。
	ERRCODE_KM_TIMEOUT	0x80001266	km 模块逻辑计算超时。
cipher	ERRCODE_CIPHER_INVALID_PARAMETER	0x80001281	参数错误。
	ERRCODE_CIPHER_UNSUPPORTED	0x80001282	当前算法不支持或当前应用场景不支持。
	ERRCODE_CIPHER_ENV_NOT_READY	0x80001284	环境未准备好，一般是函数调用顺序出错。
	ERRCODE_CIPHER_BLOCK_BUSY	0x80001286	当前使用的 cipher handle 数量已经超出最大数量。
	ERRCODE_CIPHER_TIMEOUT	0x80001287	cipher 子模块逻辑计算超时。

宏定义		错误码	描述
	ERRCODE_CIPHER_FAILED_MEM	0x80001283	内存不够用, malloc 申请内存失败。
	ERRCODE_CIPHER_SIZE_NOT_ALIGNED	0x80001285	cipher 计算数据长度未 block_size 对齐, 或 dma copy 数据长度未 32 字节对齐。
hash	ERRCODE_HASH_INVALID_PARAMETER	0x80000081	参数错误。
	ERRCODE_HASH_UNSUPPORTED	0x80000082	当前算法不支持或当前应用场景不支持。
	ERRCODE_HASH_BUSY	0x80000084	当前使用的 hash handle 数量已经超出最大数量。
	ERRCODE_HASH_TIMEOUT	0x80000085	hash 子模块逻辑计算超时。
	ERRCODE_HASH_FAILED_MEM	0x80000083	内存不够用, malloc 申请内存失败。
rsa	ERRCODE_PKE_INVALID_PARAMETER	0x80001241	参数错误。
	ERRCODE_PKE_ENV_NOT_READY	0x80001244	环境未准备好, 一般是函数调用顺序出错。
	ERRCODE_PKE_TIMEOUT	0x80001246	pke 模块逻辑计算超时。
	ERRCODE_PKE_UNSUPPORTED	0x80001242	当前算法不支持或当前应用场景不支持。
	ERRCODE_PKE_INVALID_PADDING	0x80001243	RSA 解密 hash padding 错误。
trng	ERRCODE_TRNG_INVALID_PARAMETER	0x80001231	参数错误。
	ERRCODE_TRNG_TIMEOUT	0x80001232	trng 计算超时。

3 数据类型说明

- 3.1 KM
- 3.2 AES
- 3.3 HASH
- 3.4 RSA

3.1 KM

3.1.1 drv_klad_attr_t

【描述】

klad 句柄属性。

【定义】

```
typedef struct {  
    klad_config_t klad_cfg;  
    klad_key_config_t key_cfg;  
    klad_key_secure_config_t key_sec_cfg;  
} drv_klad_attr_t;
```

【成员】

成员名	描述
klad_cfg	klad 根密钥配置，使用硬件 key 时将会判断该属性。
key_cfg	密钥属性配置。
key_sec_cfg	密钥安全属性配置。

【注意】

无。

3.1.2 drv_klad_config_t

【描述】

定义 klad 根密钥属性。

【定义】

```
typedef struct {  
    uint32_t rootkey_type;  
} drv_klad_config_t;
```

【成员】

成员名	描述
rootkey_type	密钥派生使用的根密钥类型。

【注意】

无。

3.1.3 drv_klad_clear_key_t

【描述】

明文密钥属性配置。

【定义】

```
typedef struct {  
    klad_hmac_type_t hmac_type;  
    uint32_t key_size;  
    uint8_t *key;  
    klad_key_parity_t key_parity;  
} drv_klad_clear_key_t;
```

【成员】

成员名	描述
hmac_type	hmac 类型，只有向 hmac 送密钥时才需指定。

成员名	描述
key_size	明文密钥长度，对于 cipher 模块只能为 16Byte 或 32Byte；对于 hmac 模块，则不应该大于对应 hmac 类型的块大小。
key	明文密钥数据内容。
key_parity	密钥奇偶属性配置。

【注意】

无。

3.2 AES

3.2.1 cipher_work_mode_t

【描述】

cipher 工作模式。

【定义】

```
typedef enum {
    CIPHER_WORK_MODE_ECB,
    CIPHER_WORK_MODE_CBC,
    CIPHER_WORK_MODE_CFB,
    CIPHER_WORK_MODE_OFB,
    CIPHER_WORK_MODE_CTR,
    CIPHER_WORK_MODE_CCM,
    CIPHER_WORK_MODE_GCM,
    CIPHER_WORK_MODE_CBC_MAC,
    CIPHER_WORK_MODE_CMAC,
    CIPHER_WORK_MODE_MAX,
    CIPHER_WORK_MODE_INVALID = 0xffffffff,
} drv_cipher_work_mode_t;
```

【成员】

成员名	描述
CIPHER_WORK_MODE_ECB	电子密码本模式，不安全算法，不建议使

成员名	描述
	用。
CIPHER_WORK_MODE_CBC	密码块链接模式。
CIPHER_WORK_MODE_CFB	密文反馈模式。
CIPHER_WORK_MODE_OFB	输出反馈模式。
CIPHER_WORK_MODE_CTR	计数器模式。
CIPHER_WORK_MODE_CCM	CBC_MAC + CTR。
CIPHER_WORK_MODE_GCM	GMAC + CTR。
CIPHER_WORK_MODE_CBC_MAC	AES-CBC-MAC。
CIPHER_WORK_MODE_CMAC	AES-CMAC。

【注意】

上述模式中当前支持 ECB/CBC/OFB/CTR/CCM/GCM 共 6 种工作模式，其余模式不支持，只为保持风格统一。

3.2.2 cipher_config_aes_t**【描述】**

aes 算法配置参数。

【定义】

```
typedef struct {
    cipher_key_length_t key_len;
    cipher_key_parity_t key_parity;
    cipher_bit_width_t bit_width;
    cipher_iv_change_type_t iv_change_flag;
    uint8_t iv[CIPHER_AES_IV_LEN_IN_BYTES];
} drv_cipher_config_aes_t;
```

【成员】

成员名	描述
key_len	密钥长度。

成员名	描述
key_parity	密钥奇偶属性。
bit_width	加解密数据位宽。
iv_change_flag	普通模式的 IV 配置类型。
iv	对 CBC/CTR/OFB/CFB 模式为初始（获取）的 IV 值。

【注意】

无。

3.2.3 cipher_config_aes_gcm_t

【描述】

AES GCM 加密控制信息结构。

【定义】

```
typedef struct {
    cipher_key_length_t key_len;
    cipher_key_parity_t key_parity;
    cipher_gcm_iv_change_type_t iv_change_flag;
    uint8_t *iv;
    uint32_t iv_len;
    uint8_t *add;
    uint32_t add_len;
    uint32_t data_len;
    uint8_t j0[CIPHER_AES_IV_LEN_IN_BYTES];
    uint8_t iv_ctr[CIPHER_sAES_IV_LEN_IN_BYTES];
    uint8_t iv_ghash[CIPHER_AES_IV_LEN_IN_BYTES];
} drv_cipher_config_aes_gcm;
```

【成员】

成员名	描述
key_len	密钥长度。
key_parity	密钥奇偶属性。
iv_change_flag	GCM 模式的 IV 配置类型。

成员名	描述
iv	初始 IV 值。
iv_len	GCM IV 长度。
add	补充校验数据。
add_len	补充校验数据长度。
data_len	已经执行加解密操作的数据长度，初始值为 0。
j0	内部计算值 j0，可选配置；当 iv_change_flag 取值为 CIPHER_GCM_IV_CHANGE_UPDATE 时，必须配置。
iv_ctr	内部 CTR 计算使用的 IV 值，可选配置；当 iv_change_flag 取值为 CIPHER_GCM_IV_CHANGE_UPDATE 时，必须配置。
iv_ghash	内部 GHASH 计算使用的 IV 值，可选配置；当 iv_change_flag 取值为 CIPHER_GCM_IV_CHANGE_UPDATE 时必须配置。

【注意】

无。

3.2.4 cipher_config_aes_ccm_t

【描述】

AES CCM 加密控制信息结构。

【定义】

```
typedef struct {
    cipher_key_length_t key_len;
    cipher_key_parity_t key_parity;
    cipher_ccm_iv_change_type_t iv_change_flag;
    uint32_t iv_len;
    uint8_t *add;
    uint32_t add_len;
    uint8_t tag_len;
    uint32_t total_len;
    uint8_t s0[CIPHER_AES_IV_LEN_IN_BYTES];
    uint8_t iv_ctr[CIPHER_AES_IV_LEN_IN_BYTES];
    uint8_t iv_mac[CIPHER_AES_IV_LEN_IN_BYTES];
}
```

```
} drv_cipher_config_aes_ccm_t;
```

【成员】

成员名	描述
key_len	密钥长度。
key_parity	密钥奇偶属性。
iv_change_flag	CCM 模式的 IV 配置类型。
iv_len	CCM IV 长度，取值 7 ~ 13。
add	补充校验数据。
add_len	补充校验数据长度。
tag_len	tag 长度取值 4, 6, 8, 10, 12, 14, 16。
total_len	加密数据总长度，应与后续加密的数据总长度相同，否则会报错。
s0	内部计算值 s0，可选配置；当 iv_change_flag 取值为 CIPHER_CCM_IV_CHANGE_UPDATE 时，必须配置。
iv_ctr	内部 CTR 计算使用的 IV 值，可选配置；当 iv_change_flag 取值为 CIPHER_CCM_IV_CHANGE_UPDATE 时，必须配置。
iv_mac	内部 CBC-MAC 计算使用的 IV 值，可选配置；当 iv_change_flag 取值为 CIPHER_CCM_IV_CHANGE_UPDATE 时，必须配置。

【注意】

无。

3.3 HASH

3.3.1 drv_cipher_hash_type_t

【描述】

hash 算法类型。

【定义】

```
typedef enum {
    CIPHER_HASH_TYPE_SHA1 = 0x00,
    CIPHER_HASH_TYPE_SHA224,
    CIPHER_HASH_TYPE_SHA256,
    CIPHER_HASH_TYPE_SHA384,
    CIPHER_HASH_TYPE_SHA512,
    CIPHER_HASH_TYPE_SM3 = 0x10,
    CIPHER_HASH_TYPE_HMAC_SHA1 = 0x20,
    CIPHER_HASH_TYPE_HMAC_SHA224,
    CIPHER_HASH_TYPE_HMAC_SHA256,
    CIPHER_HASH_TYPE_HMAC_SHA384,
    CIPHER_HASH_TYPE_HMAC_SHA512,
    CIPHER_HASH_TYPE_HMAC_SM3 = 0x30,
    CIPHER_HASH_TYPE_MAX,
    CIPHER_HASH_TYPE_INVALID = 0xffffffff,
} drv_cipher_hash_type_t;
```

【成员】

成员名	描述
CIPHER_HASH_TYPE_SHA1	sha1 算法。
CIPHER_HASH_TYPE_SHA224	sha224 算法。
CIPHER_HASH_TYPE_SHA256	sha256 算法。
CIPHER_HASH_TYPE_SHA384	sha384 算法。
CIPHER_HASH_TYPE_SHA512	sha512 算法。
CIPHER_HASH_TYPE_SM3	sm3 算法。
CIPHER_HASH_TYPE_HMAC_SHA1	hmac sha1 算法。
CIPHER_HASH_TYPE_HMAC_SHA224	hmac sha224 算法。
CIPHER_HASH_TYPE_HMAC_SHA256	hmac sha256 算法。
CIPHER_HASH_TYPE_HMAC_SHA384	hmac sha384 算法。
CIPHER_HASH_TYPE_HMAC_SHA512	hmac sha512 算法。
CIPHER_HASH_TYPE_HMAC_SM3	hmac sm3 算法。

【注意】

上述类型中当前只支持 SHA256，其余算法不支持，只为保持风格统一。

3.3.2 drv_cipher_hash_attr_t

【描述】

hash 句柄属性。

【定义】

```
typedef struct {
    cipher_hash_type_t hash_type;
    uint32_t keyslot;
} drv_cipher_hash_attr_t;
```

【成员】

成员名	描述
hash_type	hash 算法类型。
keyslot	keyslot 句柄 hmac 计算时会验证该属性，只计算 hash 不需要该属性。

【注意】

无。

3.3.3 drv_cipher_buf_attr_t

【描述】

存储加解密数据的缓冲区属性。

【定义】

```
typedef struct{
    uint8_t* address;
    cipher_buffer_secure_t buf_sec;
} drv_cipher_buf_attr_t;
```

【成员】

成员名	描述
address	缓冲区地址，应 4 字节对齐。
buf_sec	缓冲区安全属性。

【注意】

无。

3.4 RSA

3.4.1 drv_pke_len_t

【描述】

公钥算法模块使用的通的密钥长度。

【定义】

```
typedef enum {  
    PKE_LEN_256 = 32,  
    PKE_LEN_384 = 48,  
    PKE_LEN_512 = 64,  
    PKE_LEN_521 = 68,  
    PKE_LEN_2048 = 256,  
    PKE_LEN_3072 = 384,  
    PKE_LEN_4096 = 512,  
    PKE_LEN_MAX,  
    PKE_LEN_INVALID = 0xffffffff,  
} drv_pke_len_t;
```

【成员】

每个枚举代表的密钥长度如枚举值定义所示。

【注意】

上述长度中支持 PKE_LEN_2048 和 PKE_LEN_4096，其余不支持。

3.4.2 drv_pke_data_t

【描述】

PKE 模块数据结构体。

【定义】

```
typedef struct {
    uint32_t length;
    uint8_t *data;
} drv_pke_data_t;
```

【成员】

成员名	描述
length	数据字节长度。
data	数据地址。

【注意】

无。

3.4.3 drv_pke_rsa_scheme_t

【描述】

rsa 算法支持的模式。

【定义】

```
typedef enum {
    PKE_RSA_SCHEME_PKCS1_V15 = 0x00, /* PKCS#1 V15 */
    PKE_RSA_SCHEME_PKCS1_V21, /* PKCS#1 V21, PSS for signing, OAEP for
encryption */
    PKE_RSA_SCHEME_MAX,
    PKE_RSA_SCHEME_INVALID = 0xffffffff,
} drv_pke_rsa_scheme_t;
```

【成员】

成员名	描述
PKE_RSA_SCHEME_PKCS1_V15	PKCS#1 V15。
PKE_RSA_SCHEME_PKCS1_V21	PKCS#1 V21, PSS 用于签名, OAEP 用于加密。

【注意】

无。

3.4.4 drv_pke_rsa_pub_key_t

【描述】

RSA 算法公钥结构体。

【定义】

```
typedef struct {
    uint8_t *n;          /* point to public modulus */
    uint8_t *e;          /* point to public exponent */
    uint16_t len;        /* length of public modulus, max value is 512Byte */
} drv_pke_rsa_pub_key_t;
```

【成员】

成员名	描述
n	公共模数。
e	公共指数。
len	公共模数大小，最大为 512Byte。

【注意】

无。

3.4.5 drv_pke_rsa_priv_key_t

RSA 算法私钥结构体。

【定义】

```
typedef struct {
    uint8_t *n;
    uint8_t *e;
    uint8_t *d;
    uint8_t *p;
    uint8_t *q;
    uint8_t *dp;
```

```
uint8_t *dq;  
uint8_t *qp;  
uint16_t n_len;  
uint16_t e_len;  
uint16_t d_len;  
uint16_t p_len;  
uint16_t q_len;  
uint16_t dp_len;  
uint16_t dq_len;  
uint16_t qp_len;  
} drv_pke_rsa_priv_key_t;
```

【成员】

成员名	描述
n	公共模数。
e	公共指数。
d	私有指数。
p	第一素因数。
q	第二素因数。
dp	$D \% (P-1)$ 。
dq	$D \% (Q-1)$ 。
qp	$1 / (Q \% P)$ 。
n_len	公共模数 n 缓冲区大小，仅支持 2048bit 和 4096bit，具体内容请参考 #pke_len_t。
e_len	公共指数 e 缓冲区大小。
d_len	私有指数 d 缓冲区大小。
p_len	第一个素因数 p 的长度，应为 n_len 的一半。
q_len	第二个素因数 q 的长度，应为 n_len 的一半。
dp_len	$D \% (P-1)$ 的长度，应为 n_len 的一半。
dq_len	$D \% (Q-1)$ 的长度，应为 n_len 的一半。

成员名	描述
qp_len	1 / (Q%P)的长度，应为 n_len 的一半。

【注意】

无。